MODULE 20 · DOMAIN 2 - CONTAINER ORCHESTRATION

Storage

Volumes, persistent volumes and claims, StorageClasses, access modes, and CSI.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 20 of 35

admin@sthithapragna.com

Containers are disposable. **Data must not be.** Volumes separate data from the container's lifetime.

How long does the data live?

WHERE DATA IS WRITTEN	LIVES UNTIL	TYPICAL USE
The container's writable layer	The container is replaced	Nothing you want to keep
emptyDir volume	The Pod is deleted	Scratch space, caches, sharing between containers
hostPath volume	The node is rebuilt	Node agents only; a security risk for apps
configMap, secret volumes	The object changes	Configuration, from module 9
PersistentVolumeClaim	You delete the storage	Databases, uploads, anything to keep

ON THE EXAM

Data that must outlive the Pod needs a PersistentVolumeClaim.

emptyDir versus a persistent volume

EMPTYDIR

Tied to the Pod

- Created empty when the Pod starts
- Shared by the Pod's containers
- Deleted with the Pod
- Can live in memory: medium: Memory

PERSISTENTVOLUMECLAIM

Independent of the Pod

- Storage outside any one Pod
- Survives Pod deletion and rescheduling
- Reattached to replacement Pods
- Deleted only by its reclaim policy

THE TIEBREAKER

Container restarts keep emptyDir data; Pod deletion does not. Only persistent storage survives both.

Three objects behind persistent storage

OBJECT	SCOPE	WHAT IT IS
PersistentVolume (PV)	Cluster	A real piece of storage, such as one cloud disk
PersistentVolumeClaim (PVC)	Namespace	An app's request: size, access mode, class
StorageClass	Cluster	A kind of storage, and how to create it on demand

ON THE EXAM

Apps ask with claims; claims bind one-to-one with volumes; classes create volumes when asked.

An analogy: booking a hotel room

- **The StorageClass is the room type:** standard, or suite with a sea view.
- **The PVC is your booking:** one suite, for two nights. You never pick the room.
- **The PV is the actual room** you are given, and nobody else gets it while you have it.
- **The provisioner is a builder** who constructs a new room the moment you book. That is dynamic provisioning.

> ON THE EXAM
>
> You book a type of room; the hotel assigns an actual room. Claims are bookings, volumes are rooms.

Dynamic provisioning, step by step

01
Claim
A PVC asks for 20Gi of class fast-ssd.

›

02
Provisioner
That class's CSI driver sees the claim.

›

03
Disk
It creates a real disk in the cloud or storage system.

›

04
Bind
A PV is created and bound to the PVC.

›

05
Mount
The kubelet mounts it into the Pod.

ON THE EXAM

With dynamic provisioning, nobody creates PVs by hand. Omit storageClassName and the default class is used.

Quayside's manifest archive, stored properly

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata: {name: manifest-archive, namespace: quayside}
4  spec:
5    storageClassName: fast-ssd
6    accessModes: [ReadWriteOnce]
7    resources: {requests: {storage: 20Gi}}
8  ---
9  # the Pod refers to the claim by name:
10 volumes: [{name: archive, persistentVolumeClaim: {claimName: manifest-archive}}]
```

5-7 Which class, how it may be mounted, and how big.

10 The Pod names the claim, never the volume behind it.

THE TAKEAWAY

Module 1's lost uploads, fixed: data now lives in a claim, not the writable layer.

Access modes

MODE	SHORT	MEANS
ReadWriteOnce	RWO	Read-write by one node at a time
ReadOnlyMany	ROX	Read-only by many nodes
ReadWriteMany	RWX	Read-write by many nodes. Needs a shared filesystem
ReadWriteOncePod	RWOP	Read-write by exactly one Pod. Stable since 1.29

ON THE EXAM

ReadWriteOnce means one node, not one Pod. For a single Pod, use ReadWriteOncePod.

What a StorageClass sets

FIELD	CONTROLS
`provisioner`	Which CSI driver creates the volumes
`parameters`	Driver options, such as disk type or IOPS
`reclaimPolicy`	Delete (default) or Retain, when the claim is deleted
`volumeBindingMode`	Immediate, or WaitForFirstConsumer
`allowVolumeExpansion`	Whether claims can be grown later

ON THE EXAM

One class can be the default, marked with the annotation storageclass.kubernetes.io/is-default-class.

Reclaim policy: Delete versus Retain

DELETE

Claim gone, data gone

- The default for dynamic provisioning
- Deleting the PVC deletes the PV
- And the real disk behind it
- Right for disposable data

RETAIN

Claim gone, data kept

- The PV becomes Released
- The disk and its data remain
- An admin must clean up or reuse it
- Right for precious data

THE TIEBREAKER

With the default class, deleting a claim deletes the disk. Precious data wants Retain, and backups.

Immediate versus WaitForFirstConsumer

IMMEDIATE

Create the disk now

- Disk made as soon as the PVC exists
- Zone chosen before the Pod is placed
- Pod may need a node in another zone
- Result: Pending, volume node affinity conflict

WAITFORFIRSTCONSUMER

Wait for the Pod

- Nothing made until a Pod uses the PVC
- Scheduler places the Pod first
- Disk created in that node's zone
- The safe choice for zonal disks

THE TIEBREAKER

Zonal disks cannot follow a Pod to another zone. Let the Pod choose first: WaitForFirstConsumer.

CSI: the Container Storage Interface

- **A standard for storage plugins,** like CRI for runtimes and CNI for networks.
- **Drivers are maintained by vendors,** outside Kubernetes itself, and released on their own schedule.
- **A controller part** creates and attaches volumes; **a node part**, usually a DaemonSet, mounts them.
- **The old in-tree cloud volume plugins** have moved to CSI drivers.

> ON THE EXAM
>
> CRI, CNI, CSI: runtimes, networks, storage. Three plug-in standards.

Storage features and projects

- **Expansion:** raise a claim's size if the class allows it. Shrinking is not supported.
- **Snapshots:** VolumeSnapshot objects capture a point-in-time copy, through CSI.
- **Protection:** a claim still used by a Pod is not removed until that Pod is gone.
- **Rook**, CNCF graduated, runs Ceph storage inside the cluster as an operator. **Longhorn** is another CNCF option.

> ON THE EXAM
>
> Rook is an operator: it runs a storage system inside Kubernetes. CSI drivers connect to one.

> Quayside's web front end runs 4 replicas across different nodes. Every replica must read and write the same folder of uploaded ferry photos.
>
> The photos must survive Pod replacement.

What storage fits?

A One ReadWriteOnce claim, mounted by all four Pods

B An emptyDir volume in each of the four Pods

C A hostPath volume pointing at /data on each node

D A ReadWriteMany claim on a shared-filesystem class

◆ PAUSE AND DECIDE

What storage fits?

A One ReadWriteOnce claim, mounted by all four Pods

One node at a time: Pods on other nodes cannot mount it.

B An emptyDir volume in each of the four Pods

Four separate folders, each deleted with its Pod.

C A hostPath volume pointing at /data on each node

Different data on every node, and a security risk.

D A ReadWriteMany claim on a shared-filesystem class

Many nodes, read-write, shared, and persistent.

THE SIGNAL

Many Pods on many nodes, all writing, sharing: ReadWriteMany.

> A new Pod with a PVC stays Pending: volume node affinity conflict. The disk was created in zone a as soon as the PVC appeared.
>
> Only nodes in zone b have spare capacity. The StorageClass uses volumeBindingMode: Immediate.

What change prevents this in future?

A Set the StorageClass to WaitForFirstConsumer

B Switch the claim's access mode to ReadWriteMany

C Increase the size requested by the claim

D Remove any node affinity rules from the Pod

◆ PAUSE AND DECIDE

What change prevents this in future?

A Set the StorageClass to WaitForFirstConsumer
The Pod is placed first; the disk is then made in its zone.

B Switch the claim's access mode to ReadWriteMany
Changes sharing, not the zone the disk lives in.

C Increase the size requested by the claim
A bigger disk in the wrong zone is still in the wrong zone.

D Remove any node affinity rules from the Pod
The affinity comes from the zonal volume, not the Pod spec.

THE SIGNAL

'Volume node affinity conflict' with Immediate binding: switch to WaitForFirstConsumer.

> An engineer deletes a PVC in a test namespace by mistake. Its volume was created dynamically by the cluster's default StorageClass.
>
> Nobody changed that class's reclaim policy, and there are no backups.

What has most likely happened to the data?

A. It is kept in a Released PV, ready to rebind

B. The PV and its disk were deleted with the claim

C. It was moved into an emptyDir for safekeeping

D. The kubelet saved a snapshot before deletion

◆ PAUSE AND DECIDE

What has most likely happened to the data?

A It is kept in a Released PV, ready to rebind
Only with Retain. The default for dynamic volumes is Delete.

B The PV and its disk were deleted with the claim
Default reclaim policy Delete: claim gone, disk gone.

C It was moved into an emptyDir for safekeeping
Kubernetes never moves data between volume types.

D The kubelet saved a snapshot before deletion
Snapshots are only taken when you ask for them.

THE SIGNAL

Default class, dynamic volume, claim deleted: the data is gone. Use Retain and backups for anything precious.

> A Quayside job converts ferry videos. Each Pod needs fast scratch space for temporary files while it works.
>
> The files are useless afterwards and must not be shared with other Pods or nodes.

Which volume fits?

A. A PersistentVolumeClaim per Pod, with Retain

B. A hostPath volume pointing at /tmp on the node

C. An emptyDir volume, in memory if space allows

D. A ConfigMap mounted as a writable folder

◆ PAUSE AND DECIDE

Which volume fits?

A A PersistentVolumeClaim per Pod, with Retain

Keeps useless files forever, and costs disks.

B A hostPath volume pointing at /tmp on the node

Shared with everything on the node, and a security risk.

C An emptyDir volume, in memory if space allows

Private to the Pod, fast, and cleaned up automatically.

D A ConfigMap mounted as a writable folder

ConfigMap volumes are read-only configuration.

THE SIGNAL

Per-Pod scratch space, thrown away afterwards: emptyDir.

Module 20 in five lines

- **emptyDir** dies with the Pod; a **PersistentVolumeClaim** outlives it.
- **PVC** is the request, **PV** the storage, **StorageClass** the kind, created on demand.
- **RWO** is one node, **RWX** many nodes, **RWOP** one Pod.
- Dynamic volumes default to **Delete**; use **Retain** for precious data.
- Zonal disks: **WaitForFirstConsumer**. Plugins speak **CSI**.

That completes the building blocks. Next: **troubleshooting**, when any of them goes wrong.